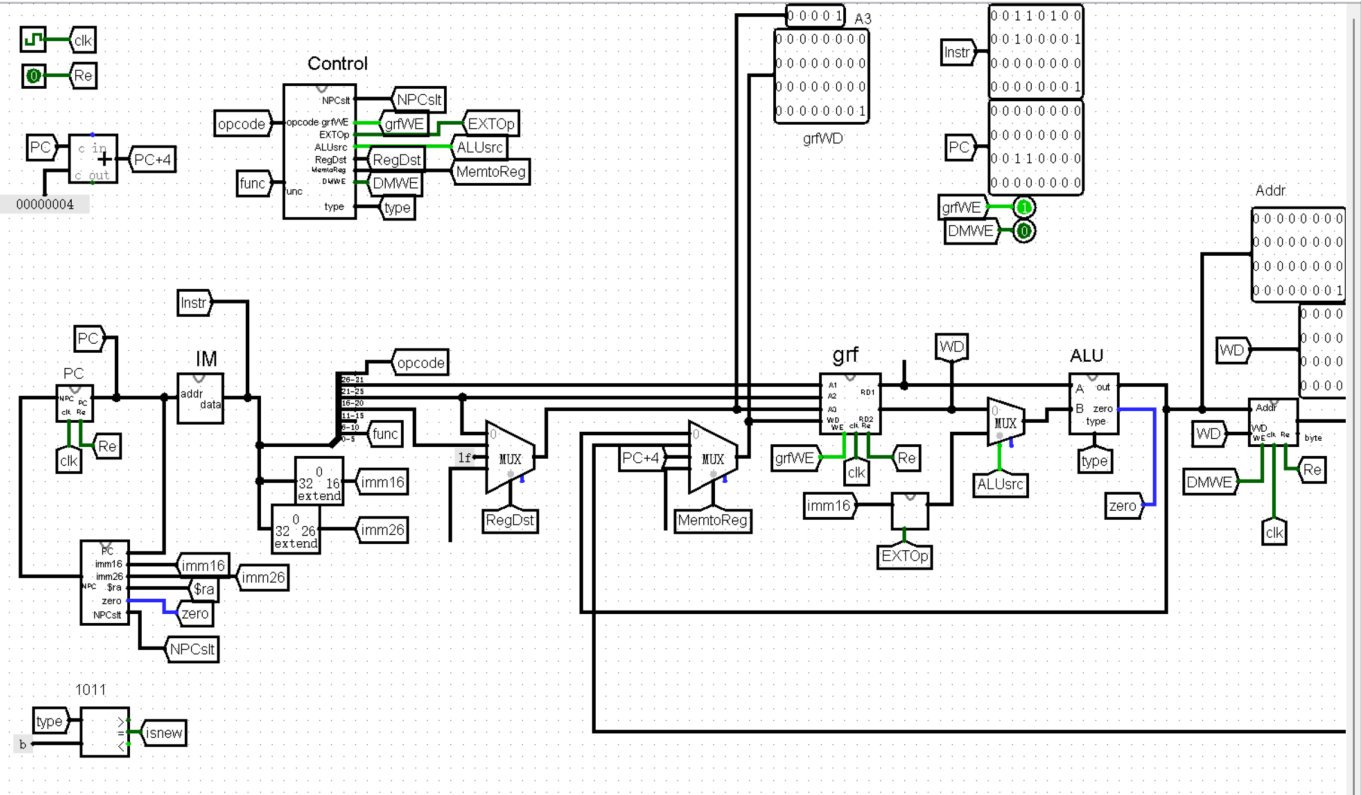


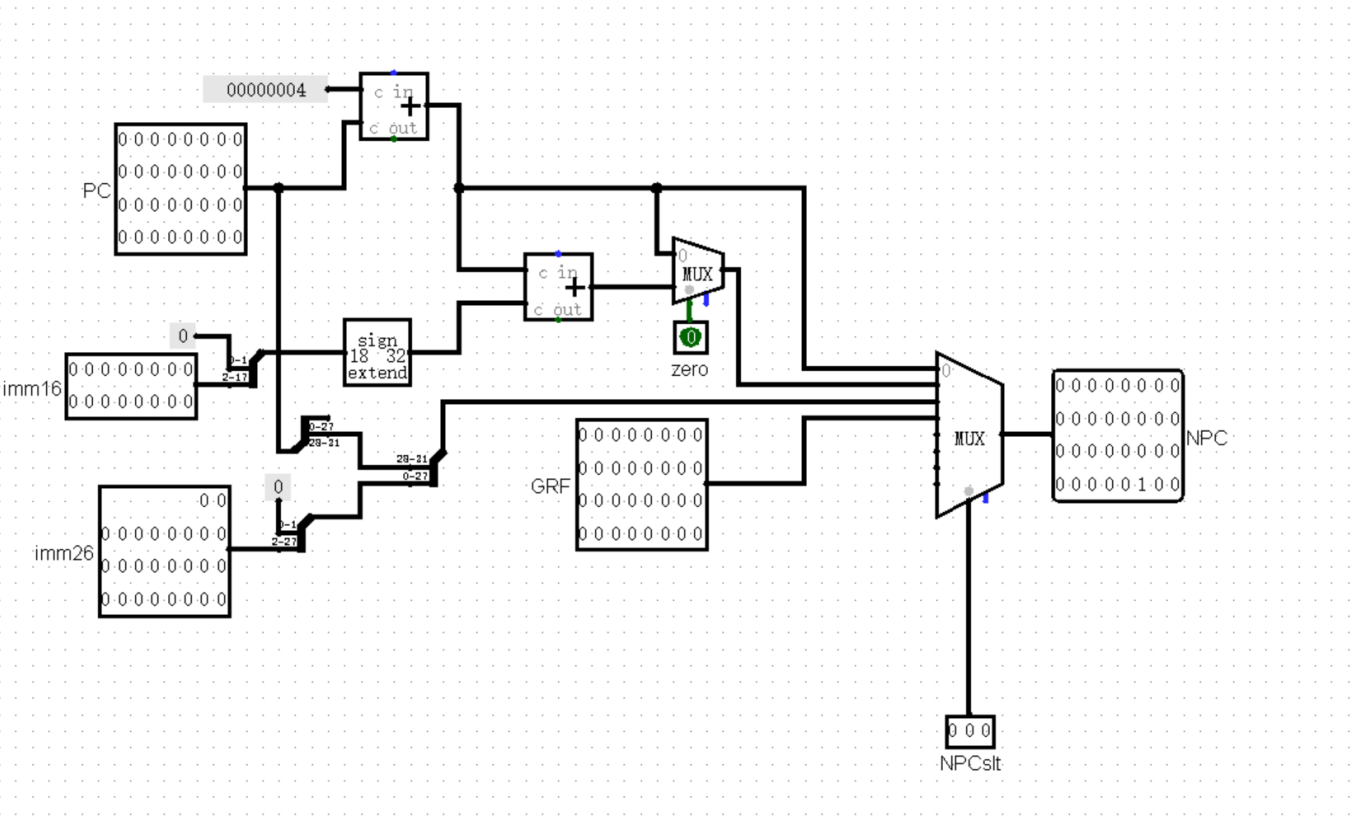
P4实验报告

总体设计概述

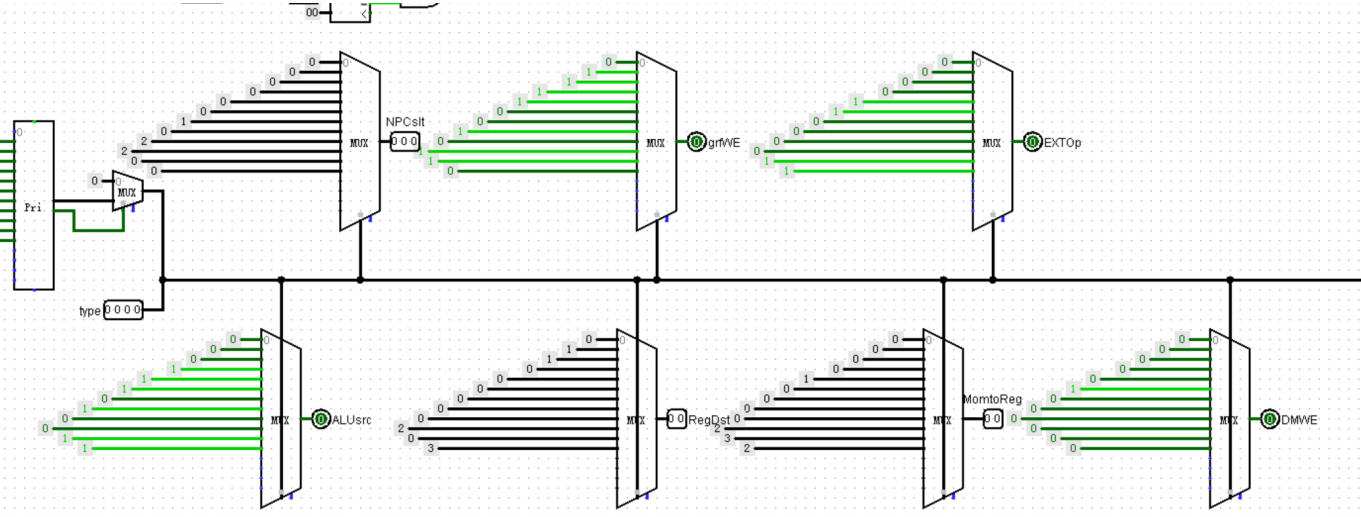
实现的指令集：add, sub, ori, lw, sw, beq, lui, jal, jr, nop MAIN：



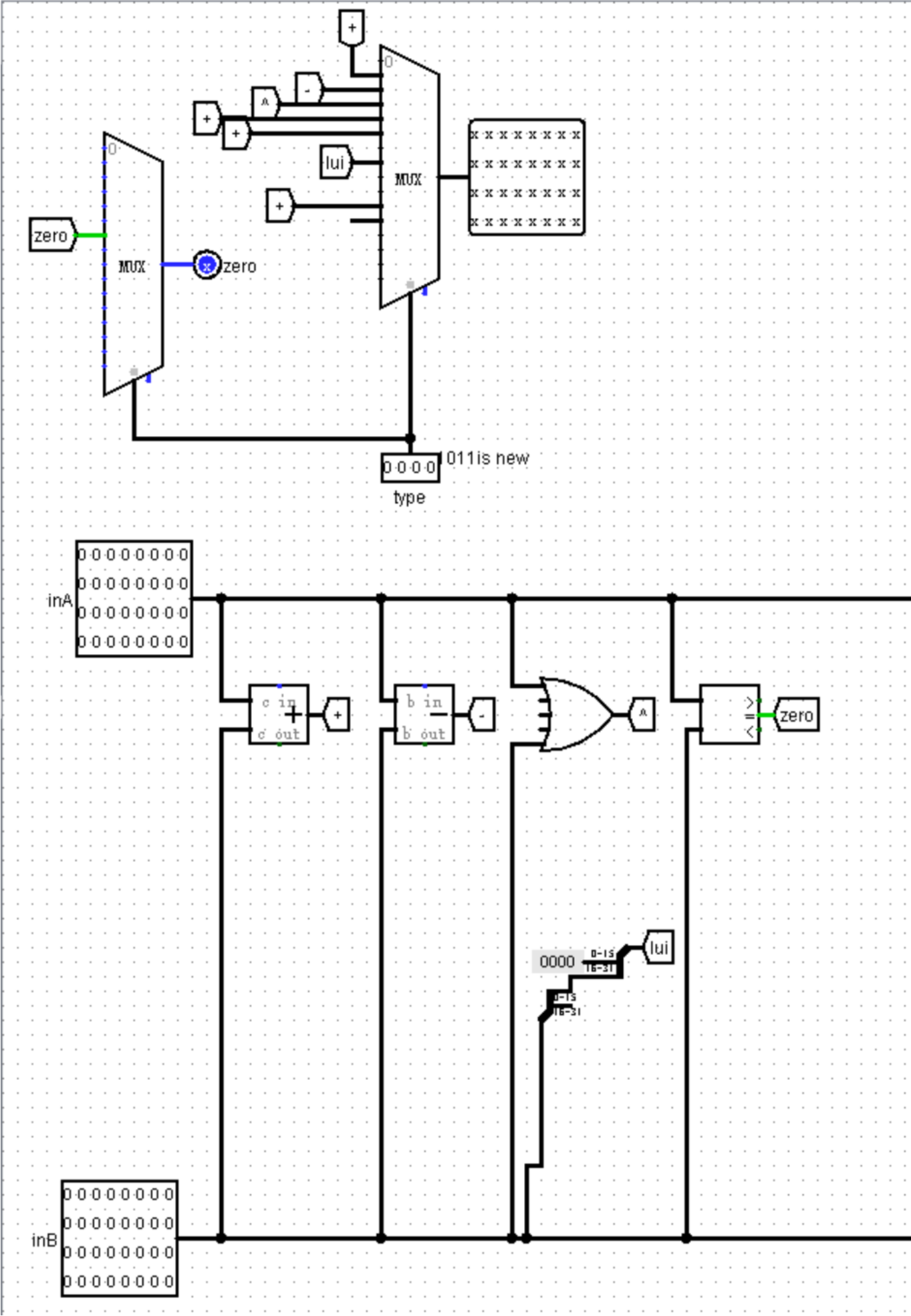
NPC:



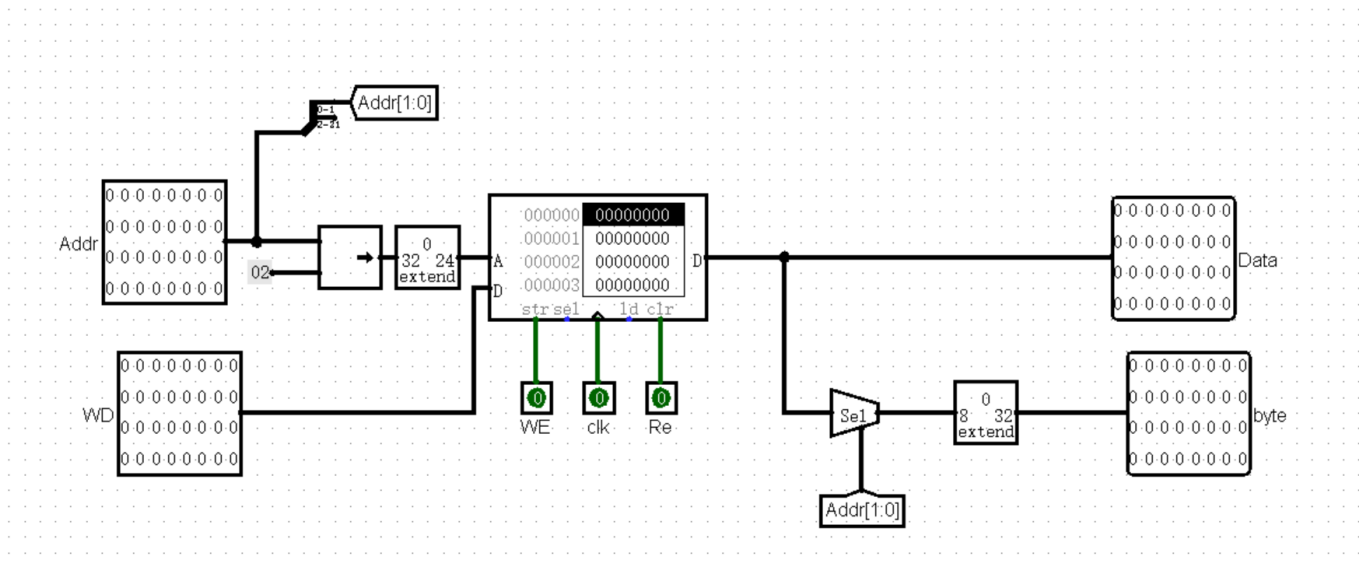
control:



ALU:



DM:



```
.text
ori $1,$1,1
ori $2,$2,2
ori $3,$3,3
ori $4,$4,1`
```

```
.text
ori $1,$1,256
ori $2,$2,1205
ori $3,$3,1616
ori $4,$4,1
```

```
//P4test_code
.text
ori $28, $0, 0
ori $29, $0, 0
ori $1, $0, 13398
add $1, $1, $1
lw $1, 4($0)
sw $1, 4($0)
lui $2, 30840
sub $3, $2, $1
lui $5, 4660
ori $4, $0, 5
nop
sw $5, -1($4)
lw $3, -1($4)
beq $3, $5, label1
nop
beq $0, $0, label2
nop
label1:
```

```
ori $7, $3, 1028
beq $7, $3, label2
nop
lui $8, 30583
ori $8, $8, 65535
sub $0, $0, $8
ori $0, $0, 4352
add $10, $7, $6
ori $8, $0, 0
ori $9, $0, 1
ori $10, $0, 1
label3:
add $8, $8, $10
beq $8, $9, label3
jal label4
nop
add $10, $10, $10
label2:
beq $0, $0, label2
label4:
add $10, $10, $10
jr $31
nop
```

关键信号声明

NPCslt[2:0] 0:PC+4 1:PC+4+imm16 2:PC+imm26 3:\$ra **grfWE** 决定是否回写入grf **EXTOp** 0: 无符号 1: 符号扩展 **ALUsrc** 决定ALU中的inputB输入: 0:rt寄存器中的值 1:imm16 **RegDst** 回写到哪个寄存器? 0: rt 1: rd 2: 31号

MemtoReg 0: 非lw的回写 1: lw回写 2: PC+4 **DMWE** 决定DM的写使能 **type** 1add, 2sub, 3ori, 4lw, 5sw, 6beq, 7lui, 8jal, 9jr

关键子模块

GRF:

信号名称	方向	功能描述
A1[4:0]	输入	5位地址输入信号，指定要读取数据的寄存器，数据输出到RD1
A2[4:0]	输入	5位地址输入信号，指定要读取数据的寄存器，数据输出到RD2
A3[4:0]	输入	5位地址输入信号，指定要写入数据的目标寄存器
RD1[31:0]	输出	输出A1指定寄存器中的32位数据
RD2[31:0]	输出	输出A2指定寄存器中的32位数据
WD[31:0]	输入	32位数据输入信号，是要写入目标寄存器的待存数据
GRFWrEn	输入	写使能信号; 1表示写入有效, 0表示写入无效
clk	输入	时钟信号，控制寄存器堆的读写时序

信号名称	方向	功能描述
reset	输入	异步复位信号；1时将32个寄存器的数据全部清零，0时无效

IFU:

信号名称	方向	功能描述
NPC[31:0]	输入	待写入PC的下一条指令地址
clk	输入	时钟信号，控制PC更新时序
reset	输入	异步复位信号，复位时初始化PC
PC	输出	当前正在执行的指令地址
Instr[31:0]	输出	从IM中读取的32位指令值

EXT:

控制信号值	功能描述
EXT_unsign	零扩展（高位补0）
EXT_sign	符号扩展（高位补原符号位）

ALU:

信号名称	方向	功能描述
A[31:0]	输入	32位运算数A
B[31:0]	输入	32位运算数B
ALUOp[4:0]	输入	运算控制信号，指定运算类型
shamt[4:0]	输入	移位指令的移位位数（0-31）
C[31:0]	输出	32位运算结果
overflow	输出	运算溢出指示信号（扩展功能）

NPC:

信号名称	方向	功能描述
PC[31:0]	输入	32位输入当前地址
jump	输入	指示b类型指令是否跳转
NPCOp[1:0]	输入	控制信号，指定NPC计算方式
RA[31:0]	输入	\$ra寄存器保存的32位地址（用于jalr/jr指令）
NPC[31:0]	输出	32位输出次地址
PC+4[31:0]	输出	输出PC+4的值

控制信号值	功能描述
NPC_pc4	NPC=PC+4
NPC_b	执行beq指令
NPC_j_jal	执行j、jal指令
NPC_jalr_jr	执行jalr、jr指令

DM:

信号名称	方向	功能描述
Addr[31:0]	输入	待操作的内存地址
WD[31:0]	输入	待写入内存的值
clk	输入	时钟信号
reset	输入	异步复位信号
DMWrEn	输入	写使能信号；1：写入有效；0：写入无效
DMOp[2:0]	输入	控制信号
RD[31:0]	输出	输入地址指向的内存中储存的值

思考题

1. MIPS 中以字节为单位，而在我们设计的 DM 中，每一个 `reg[31:0]` 为一个单位。Addr 来自 ALU 的输出端口，代表要读取的 DM 存储器的地址，在我们的 4KB 的 DM 设计中应当取 `[11:0]`，又因为按字节寻址，因此取 `[11:2]`。

2.

```
assign ALUOp = (addu | lw | sw) ? ALU_add : (subu) ? ALU_sub : (ori) ? ALU_or : (lui) ? ALU_lui :
4'b0000; 通过嵌套三元运算符，根据不同指令类型生成对应的 ALUOp case(ALUOp) ALU_add: C = A + B;
ALU_sub: C = A - B;
ALU_or: C = A | B; ALU_and: C = A & B;
ALU_xor: C = A ^ B; ALU_lui: C = B << 16;
default: C = 32'h0000_0000; // 默认输出32位0 endcase
```

整体功能：这部分代码是 CPU 中 ALU 模块的核心逻辑，负责根据指令类型生成控制信号（ALUOp），并通过该控制信号执行具体的算术或逻辑运算，最终输出运算结果。

- assign 三目运算符不需要自己再定义寄存器
- case 语句和 assign 都可以通过宏定义的方式使代码更加美观，增强可读性
- assign 更方便增添指令

3.异步复位中，clk、reset等级一致；同步复位中，clk的优先级高于reset 4.根据 RTL 语言描述： addi 与 addiu 的区别在于当出现溢出时， addiu 忽略溢出，并将溢出的最高位舍弃； addi 会报告 SignalException(IntegerOverflow) 故忽略溢出的时候二者是等价的。